```
#Martial Arts Course enrolment system

#- What my code is about: With it, you can view martial art courses and enrol in them by paying. You can also reserve and check the timetable of classes. You can log in as an admin or a user to complete and view activities (such as modifying courses if you're an admin or reserving classes if you're a user).

#- Involving Python, Tkinter, GUI, SQLite, Object Oriented Programming, file handling, data validation, error handling, CRUD operations, hashing, encryption, email integration, file and event handling, database

#  interaction, data security and validation, regular expressions

#- I have learnt :

# 1. Organise code into classes to improve structure and reusability.

# 2. GUI design and implementation.

# 3. Database management.

# 4. User authentication.

# 5. Input validation using regular expressions.

# 6. Data encryption using hashing.

# 7. Email communication, such as notifications.

# 8. Implicit error handling.

# - Self-reflection: This project helped me gain practical experience with GUI development. To enhance this application, in the future, I would implement more robust error handling to improve its reliability.


from tkinter import *

import tkinter as tk

import sqlite3

import hashlib

import uuid

from tkinter import messagebox

import re

from cryptography.fernet import Fernet

from email.mime.text import MIMEText

import smtplib
```

```
import datetime



########################################   REGISTRATION AND TIMETABLE
########################################

#regular expressions

"""

^                                               Match the beginning of the string

(?=.*[0-9])                                     Require that at least one digit
appear anywhere in the string

(?=.*[a-z])                                     Require that at least one lowercase
letter appear anywhere in the string

(?=.*[A-Z])                                     Require that at least one uppercase
letter appear anywhere in the string

(?=.*[*.!@$%^&(){}[]:;<>,.?/~_+-=|\])    Require that at least one special
character appear anywhere in the string

.{8,32}                                         The password must be at least 8
characters long, but no more than 32

$                                               Match the end of the string.

"""

#connection with db

conn = sqlite3.connect('program.db')

with conn:

  cursor=conn.cursor()

#querries

cursor.execute('CREATE TABLE IF NOT EXISTS  users(id TEXT PRIMARY KEY, Password
TEXT,Email TEXT, Course Text)')

conn.commit()



class TimetableApp():
```

```
    def __init__(self, root):
        self.root = root
        self.root.title("Course Timetable")
        self.courses_label = tk.Label(self.root, text="", font=("Arial", 12))
        self.courses_label.pack()

        # Connect to the database (or create it if it doesn't exist)
        self.conn = sqlite3.connect('program.db')


        # Create a cursor object
        self.cursor = self.conn.cursor()

        # Create buttons for each day of the week
        days = ["Monday", "Tuesday", "Wednesday", "Thursday", "Friday"]
        for day in days:
            button = tk.Button(self.root, text=day, command=lambda day=day:
self.show_day_courses(day))
            button.pack()


    # Create a function to retrieve the course information for a given day
    def get_day_courses(self, day):
        self.cursor.execute("SELECT * FROM schedule WHERE day=?", (day,))
        return self.cursor.fetchall()


    # Create a function to update the label when a button is clicked
    def show_day_courses(self, day):
        courses = self.get_day_courses(day)
```

```
        courses_string = "\n".join([f"{course[0]} ({course[2]} - {course[3]})"
for course in courses])
        self.courses_label.config(text=courses_string)



class RegisterForm(tk.Tk): #u do this at the beginning w tk.tk
    def __init__(self):  #the rest "self" r the attributes
        super().__init__()

        course = StringVar()
        # Set up the GUI
        self.title("Registration Form")
        self.geometry("500x500")


        # Create labels and entry fields for course, email, and password
        self.email_label = tk.Label(self, text="Email:")
        self.email_entry = tk.Entry(self)
        self.password_label = tk.Label(self, text="Password:")
        self.password_entry = tk.Entry(self, show="*")
        self.label_course = tk.Label(self, text= "Course to enroll")
        self.course_entry = tk.Entry(self, text="Choose course")
        list1 = ['Boxing','Muay Thai','Judo']
        self.droplist=OptionMenu(self,course, *list1)
        self.droplist.config(width=15)
        course.set('Select your course')
        self.droplist.place(x=190,y=175)

        # Create a register button
```

```
        self.register_button = tk.Button(self, text="Register",
command=self.register_user)
        self.login_label = tk.Label(self, text="Already have an account?")
        self.login_button = tk.Button(self, text="Login",
command=self.go_to_login)



        # Add the widgets to the GUI
        self.email_label.pack()
        self.email_entry.pack()
        self.password_label.pack()
        self.password_entry.pack()
        self.register_button.pack()
        self.login_label.pack()
        self.login_button.pack()
        self.label_course.pack()
        self.course_entry.pack()

    def register_user(self):
        # Get the inputted email and password
        entered_email = self.email_entry.get()
        entered_password = self.password_entry.get()
        entered_course = self.course_entry.get()


        def final_registeration(entered_email, entered_password):
            # Hash the entered password
            hashed_password =
hashlib.sha256(entered_password.encode()).hexdigest()
```

```
            # Connect to the SQLite database
            connection = sqlite3.connect("program.db")
            cursor = connection.cursor()

            # Insert the user information into the database
            cursor.execute("INSERT INTO USERS (id, email, password, course)
VALUES (?, ?, ?, ?)", (str(uuid.uuid4()), entered_email, hashed_password,
entered_course))
            connection.commit()

            # Show a message to confirm the registration
            tk.messagebox.showinfo("Registration Successful", "You have
successfully registered.")

            # Close the database connection
            connection.close()



        if len(entered_password)<=20
            if re.match("^(?=.*[0-9])(?=.*[a-z])(?=.*[A-Z](?=.*[^a-bA-B0-
9]))",entered_password):
                # messagebox.showwarning("User: "+ entered_email +" successfully
registered")
                final_registeration(entered_email, entered_password)
            else:
                messagebox.showwarning("User","Enter valid password")
                return False
            #regular expressions



        else:
```

```
            messagebox.showwarning("Invalid","Length try to exceed")
            return False

        if len(entered_email)>7:
            if re.match("^([a-zA-Z0-9_\-\.]+)@([a-zA-Z0-9_\-\.]+)\.([a-zA-
Z]{2,5})$",entered_email):
                return True
            else:
                messagebox.showwarning("Alert","Invalid E-mail enter by user")
                return False
        else:
            messagebox.showwarning("Alert","Email length is too small")




    def go_to_login(self):
        self.destroy()
        login_page = LoginForm() #this has to be the name of the class you want
to be redirected to
# use this code above to redirect to another page, use the class to specify what
page



########################################    LOGIN
#####################################################
class LoginForm(tk.Tk):
    def __init__(self):
        super().__init__()
```

```
        # Set up the GUI
        self.title("Login Form")
        self.geometry("500x500")

        # Create labels and entry fields for username and password
        self.email_label = tk.Label(self, text="Email:")
        self.email_entry = tk.Entry(self)
        self.password_label = tk.Label(self, text="Password:")
        self.password_entry = tk.Entry(self, show="*")

        # Create a login button
        self.login_button = tk.Button(self, text="Login",
command=self.check_credentials)


        self.register_label = tk.Label(self, text="Dont have an account?")
        self.register_button = tk.Button(self, text="Register",
command=self.go_to_register)

        # Add the widgets to the GUI
        self.email_label.pack()
        self.email_entry.pack()
        self.password_label.pack()
        self.password_entry.pack()
        self.login_button.pack()
        self.register_label.pack()
        self.register_button.pack()
```

```
    def check_credentials(self):
        # Get the inputted username and password
        entered_email = self.email_entry.get()
        entered_password = self.password_entry.get()

        # Hash the entered password
        hashed_password = hashlib.sha256(entered_password.encode()).hexdigest()

        # Connect to the SQLite database
        connection = sqlite3.connect("program.db")
        cursor = connection.cursor()

        # Query the database for the entered username and hashed password
        cursor.execute("SELECT * FROM users WHERE email=? AND password=?",
(entered_email, hashed_password))
        result = cursor.fetchone()

        # Check if the query returned a result
        if result:
            print("Login successful!")
            print(result)


            def go_to_timetable(self):
                self.destroy()
                timetable_page = TimetableApp(root)
                timetable_page.mainloop()
```

```
            #check if user is admin or normal user
            if result[4] == 'User':
                #take them to timetable page
                go_to_timetable(self)
            else:
                #take them to admin page
                pass

            # Grant access to the user

        else:
            print("Invalid username or password.")

        # Close the database connection
        connection.close()

    def go_to_register(self):
        self.destroy()
        register_page = RegisterForm()


if __name__ == "__main__":
```

```
    form = LoginForm()
    form.mainloop()


########################################    ADMIN PAGE
#####################################################


class MainWindow(tk.Tk):
  def __init__(self):
    super().__init__()
    self.entered_email = tk.StringVar()
    self.entered_password = tk.StringVar()
    self._switch_to_login_page()

  def _switch_to_login_page(self):
    for widget in self.winfo_children():
      widget.destroy()

  def _login(self):
    if self.entered_email.get() == "0ADMIN@ADMIN.COM" and
self.entered_password.get() == "0Admin":
      self._switch_to_admin_page()
    else:
      self._switch_to_non_admin_page()

  def _switch_to_admin_page(self):
    # create and display the admin page
    for widget in self.winfo_children():
      widget.destroy()
```

```
    tk.Label(self, text="Admin Page").pack()

  def _switch_to_non_admin_page(self):
    # create and display the non-admin page
    for widget in self.winfo_children():
      widget.destroy()
    tk.Label(self, text="Non-Admin Page").pack()


#########################MODIFICATION OF COURSE
#####################################################################
def create_table():
   cursor.execute('''CREATE TABLE IF NOT EXISTS schedule (name TEXT PRIMARY KEY,
day TEXT, start_time TIME, end_time TIME)''')
   conn.commit()


def add_course(course_name, day, start_time, end_time):
    cursor.execute("INSERT INTO schedule VALUES (?,?,?,?)", (course_name, day,
start_time, end_time))
    conn.commit()


# Create the main Tkinter window
root = tk.Tk()
root.title("Admin Control Panel")
create_table()


# Create a frame for the course name, day, start time, and end time inputs
input_frame = tk.Frame(root)
input_frame.pack()
```

```
# Create labels and entry widgets for the course name, day, start time, and end time
course_name_label = tk.Label(input_frame, text="Course Name:")
course_name_label.grid(row=0, column=0)
course_name_entry = tk.Entry(input_frame)
course_name_entry.grid(row=0, column=1)


day_label = tk.Label(input_frame, text="Day:")
day_label.grid(row=1, column=0)
day_entry = tk.Entry(input_frame)
day_entry.grid(row=1, column=1)


start_time_label = tk.Label(input_frame, text="Start Time:")
start_time_label.grid(row=2, column=0)
start_time_entry = tk.Entry(input_frame)
start_time_entry.grid(row=2, column=1)


end_time_label = tk.Label(input_frame, text="End Time:")
end_time_label.grid(row=3, column=0)
end_time_entry = tk.Entry(input_frame)
end_time_entry.grid(row=3, column=1)


 # Create a "Add Course" button
add_course_button = tk.Button(root, text="Add Course", command=lambda: add_course(course_name_entry.get(), day_entry.get(), start_time_entry.get(), end_time_entry.get()))
add_course_button.pack()
```

```
root.mainloop()



if __name__ == "__main__":
    app = MainWindow()
    app.mainloop()



############################################  TIMETABLE
##############################################
class MainApp:
    def __init__(self, root):
        self.root = root
        self.root.title("Main Page")

        # Create a button to show the timetable page
        button = tk.Button(self.root, text="Timetable",
command=self.show_timetable_page)
        button.pack()

    def show_timetable_page(self):
        # Clear the contents of the root window
        for widget in self.root.winfo_children():
            widget.destroy()

        # Create the timetable page
        timetable_app = TimetableApp(self.root)
```

```
class TimetableApp():

    def __init__(self, root):
        self.root = root
        self.root.title("Course Timetable")
        self.courses_label = tk.Label(self.root, text="", font=("Arial", 12))
        self.courses_label.pack()

        # Connect to the database (or create it if it doesn't exist)
        self.conn = sqlite3.connect('program.db')


        # Create a cursor object
        self.cursor = self.conn.cursor()

        # Create buttons for each day of the week
        days = ["Monday", "Tuesday", "Wednesday", "Thursday", "Friday"]
        for day in days:
            button = tk.Button(self.root, text=day, command=lambda day=day:
self.show_day_courses(day))
            button.pack()


    # Create a function to retrieve the course information for a given day
    def get_day_courses(self, day):
        self.cursor.execute("SELECT * FROM schedule WHERE day=?", (day,))
        return self.cursor.fetchall()
```

```
    # Create a function to update the label when a button is clicked
    def show_day_courses(self, day):
        courses = self.get_day_courses(day)
        courses_string = "\n".join([f"{course[0]} ({course[2]} - {course[3]})"
for course in courses])
        self.courses_label.config(text=courses_string)


root = tk.Tk()
app = MainApp(root)
root.mainloop()


######################################## RESERVATIONS
################################################################


class ReservationSystem:
    def __init__(self, master):
        self.master = master
        master.title("Reservation System")


        # create database connection
        self.conn = sqlite3.connect('program.db')
        self.c = self.conn.cursor()


        # create table to hold reservation data
        self.c.execute('''CREATE TABLE IF NOT EXISTS reservations
                          (course text, name text, email text, phone text, date
text)''')
```

```
        # create course selection menu
        self.course_label = tk.Label(master, text="Select a course:")
        self.course_label.grid(row=0, column=0)
        self.course_var = tk.StringVar(master)
        self.course_var.set("Judo")  # default value
        self.course_menu = tk.OptionMenu(master, self.course_var, "Judo", "Muay
Thai", "Boxing")
        self.course_menu.grid(row=0, column=1)

        # create input fields
        self.name_label = tk.Label(master, text="Name:")
        self.name_label.grid(row=1, column=0)
        self.name_entry = tk.Entry(master)
        self.name_entry.grid(row=1, column=1)

        self.email_label = tk.Label(master, text="Email:")
        self.email_label.grid(row=2, column=0)
        self.email_entry = tk.Entry(master)
        self.email_entry.grid(row=2, column=1)

        self.phone_label = tk.Label(master, text="Phone:")
        self.phone_label.grid(row=3, column=0)
        self.phone_entry = tk.Entry(master)
        self.phone_entry.grid(row=3, column=1)
```

```
        self.date_label = tk.Label(master, text="Date:")
        self.date_label.grid(row=4, column=0)
        self.date_entry = tk.Entry(master)
        self.date_entry.grid(row=4, column=1)

        # create submit button
        self.submit_button = tk.Button(master, text="Submit",
command=self.submit)
        self.submit_button.grid(row=5, column=1)

        # create a queue for each course
        self.judo_queue = []
        self.muay_thai_queue = []
        self.boxing_queue = []

    def submit(self):
        course = self.course_var.get()
        name = self.name_entry.get()
        email = self.email_entry.get()
        phone = self.phone_entry.get()
        date = self.date_entry.get()

        # check if input fields are not empty
        if name == '' or email == '' or phone == '' or date == '':
            return
```

```
        # insert reservation data into database
        self.c.execute("INSERT INTO reservations VALUES (?, ?, ?, ?, ?)",
(course, name, email, phone, date))
        self.conn.commit()


        # add reservation to the appropriate queue
        if course == 'Judo':
            self.judo_queue.append((name, email, phone, date))
        elif course == 'Muay Thai':
            self.muay_thai_queue.append((name, email, phone, date))
        elif course == 'Boxing':
            self.boxing_queue.append((name, email, phone, date))


        # clear input fields
        self.name_entry.delete(0, tk.END)
        self.email_entry.delete(0, tk.END)
        self.phone_entry.delete(0, tk.END)
        self.date_entry.delete(0, tk.END)


        # show success message
        success_label = tk.Label(self.master, text="Reservation successful!")
        success_label.grid(row=6, column=1)
    def enqueue(self, course, name, email, phone, date):
        # add reservation to the appropriate queue
        if course == 'Judo':
            self.judo_queue.append((name, email, phone, date))
        elif course == 'Muay Thai':
```

```
            self.muay_thai_queue.append((name, email, phone, date))
        elif course == 'Boxing':
            self.boxing_queue.append((name, email, phone, date))

    def dequeue(self, course):
        # get the next reservation from the appropriate queue
        if course == 'Judo':
            if len(self.judo_queue) > 0:
                return self.judo_queue.pop(0)
            else:
                return None
        elif course == 'Muay Thai':
            if len(self.muay_thai_queue) > 0:
                return self.muay_thai_queue.pop(0)
            else:
                return None
        elif course == 'Boxing':
            if len(self.boxing_queue) > 0:
                return self.boxing_queue.pop(0)
            else:
                return None

root = tk.Tk()
reservation = ReservationSystem(root)
root.mainloop()
```

```
############################################## ENROLLING
##############################################
from tkinter import ttk
from datetime import datetime


# Connect to the SQLite database
conn = sqlite3.connect("program.db")


# Create the transactions table if it doesn't exist
c = conn.cursor()
c.execute("""CREATE TABLE IF NOT EXISTS transactions (
            id INTEGER PRIMARY KEY,
            date TEXT,
            item TEXT,
            price REAL
            )""")
conn.commit()


# Create the users table if it doesn't exist
c.execute("""CREATE TABLE IF NOT EXISTS usr (
            id INTEGER PRIMARY KEY,
            name TEXT,
            course TEXT,
            date_enrolled TEXT
            )""")
conn.commit()
```

```
# Define global variables
courses = {
    "Judo": 99.99,
    "Muay Thai": 79.99,
    "Boxing": 69.99
}

# Define the main window
root = tk.Tk()
root.title("Enrollment System")

# Define functions for handling enrollments
def enroll_user():
    # Get the user's name and selected course from the GUI
    name = name_var.get()
    course = course_var.get()

    # Get the price of the selected course
    price = courses[course]

    # Add a transaction to the database
    date = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    c.execute("INSERT INTO transactions (date, item, price) VALUES (?, ?, ?)",
(date, course, price))
    conn.commit()

    # Add the user to the database
    date_enrolled = datetime.now().strftime("%Y-%m-%d")
```

```
    c.execute("INSERT INTO usr (name, course, date_enrolled) VALUES (?, ?, ?)", (name, course, date_enrolled))
    conn.commit()

    # Generate a receipt
    receipt_text = f"Enrollment Receipt\n\nName: {name}\nCourse: {course}\nPrice: ${price:.2f}\nDate: {date}\n\nThank you for enrolling!"
    receipt_label.config(text=receipt_text)


# Define functions for handling billing
def calculate_total():
    # Get the selected course from the GUI
    course = course_var.get()

    # Get the price of the selected course
    price = courses[course]

    # Calculate the total price based on the quantity
    quantity = int(quantity_var.get())
    total = price * quantity

    # Update the total label on the GUI
    total_label.config(text=f"Total: ${total:.2f}")


# Create labels and entry widgets for the GUI
name_label = tk.Label(root, text="Name:")
name_var = tk.StringVar()
name_entry = tk.Entry(root, textvariable=name_var)
```

```
course_label = tk.Label(root, text="Course:")
course_var = tk.StringVar()
course_dropdown = ttk.Combobox(root, textvariable=course_var,
values=list(courses.keys()), state="readonly", postcommand=calculate_total)

quantity_label = tk.Label(root, text="Quantity:")
quantity_var = tk.StringVar(value="1")
quantity_entry = tk.Entry(root, textvariable=quantity_var, validate="key",
validatecommand=(root.register(lambda s: s.isdigit() or s == ""), "%S"), width=3)
quantity_entry.bind("<KeyRelease>", lambda e: calculate_total())

total_label = tk.Label(root, text="Total: $0.00")

# Create a button for the GUI to enroll the user
enroll_button = tk.Button(root, text="Enroll", command=enroll_user)

# Create a label for the receipt
receipt_label = tk.Label(root, text="")

# Pack the widgets into the GUI
name_label.grid(row=0, column=0, padx=5, pady=5, sticky=tk.E)
name_entry.grid(row=0, column=1, padx=5, pady=5, sticky=tk.W)

course_label.grid(row=1, column=0, padx=5, pady=5, sticky=tk.E)
```

```
course_dropdown.grid(row=1, column=1, padx=5, pady=5, sticky=tk.W)

quantity_label.grid(row=2, column=0, padx=5, pady=5, sticky=tk.E)
quantity_entry.grid(row=2, column=1, padx=5, pady=5, sticky=tk.W)

total_label.grid(row=2, column=2, padx=5, pady=5, sticky=tk.E)

enroll_button.grid(row=3, column=1, padx=5, pady=5)

receipt_label.grid(row=4, column=0, columnspan=3, padx=5, pady=5)

root.mainloop()


#####################################card
transaction############################################################


class CardTransactionSystem:
    def __init__(self, master):
        self.master = master
        master.title("Card Transaction System")

        # create database connection
        self.conn = sqlite3.connect('program.db')
```

```
        self.c = self.conn.cursor()

        # create table to hold transaction data
        self.c.execute('''CREATE TABLE IF NOT EXISTS transac
                          (card_number text, amount real, date text)''')

        # create input fields
        self.card_number_label = tk.Label(master, text="Card Number:")
        self.card_number_label.grid(row=0, column=0)
        self.card_number_entry = tk.Entry(master)
        self.card_number_entry.grid(row=0, column=1)

        self.amount_label = tk.Label(master, text="Amount:")
        self.amount_label.grid(row=1, column=0)
        self.amount_entry = tk.Entry(master)
        self.amount_entry.grid(row=1, column=1)

        self.date_label = tk.Label(master, text="Date:")
        self.date_label.grid(row=2, column=0)
        self.date_entry = tk.Entry(master)
        self.date_entry.grid(row=2, column=1)

        # create submit button
        self.submit_button = tk.Button(master, text="Submit",
command=self.submit)
        self.submit_button.grid(row=3, column=1)
```

```
    def submit(self):
        card_number = self.card_number_entry.get()
        amount = float(self.amount_entry.get())
        date = self.date_entry.get()

        # insert transaction data into database
        self.c.execute("INSERT INTO transac VALUES (?, ?, ?)", (card_number,
amount, date))
        self.conn.commit()

        # clear input fields
        self.card_number_entry.delete(0, tk.END)
        self.amount_entry.delete(0, tk.END)
        self.date_entry.delete(0, tk.END)

    def __del__(self):
        # close database connection
        self.conn.close()

root = tk.Tk()
card_transaction_system = CardTransactionSystem(root)
root.mainloop()
```

File: c:\Users\gizem\NEA\system.py

```
1: from tkinter import *

2: import tkinter as tk

3: import sqlite3

4: import hashlib

5: import uuid

6: from tkinter import messagebox

7: import re

8: from cryptography.fernet import Fernet

9: from email.mime.text import MIMEText

10: import smtplib

11: import datetime

12:

13: ##########################################   REGISTRATION AND TIMETABLE
##########################################

14: #regular expressions

15: """

16: ^                                           Match the beginning of the
string

17: (?=.*[0-9])                                 Require that at least one digit
appear anywhere in the string

18: (?=.*[a-z])                                 Require that at least one
lowercase letter appear anywhere in the string

19: (?=.*[A-Z])                                 Require that at least one
uppercase letter appear anywhere in the string

20: (?=.*[*.!@$%^&(){}[]:;<>,.?/~_+-=|\])    Require that at least one special
character appear anywhere in the string

21: .{8,32}                                     The password must be at least 8
characters long, but no more than 32

22: $                                           Match the end of the string.

23: """

24: #connection with db

25: conn = sqlite3.connect('program.db')
```

```
with conn:
  cursor=conn.cursor()
#querries
cursor.execute('CREATE TABLE IF NOT EXISTS  users(id TEXT PRIMARY KEY, Password TEXT,Email TEXT, Course Text)')
conn.commit()


class TimetableApp():

    def __init__(self, root):
        self.root = root
        self.root.title("Course Timetable")
        self.courses_label = tk.Label(self.root, text="", font=("Arial", 12))
        self.courses_label.pack()

        # Connect to the database (or create it if it doesn't exist)
        self.conn = sqlite3.connect('program.db')

        # Create a cursor object
        self.cursor = self.conn.cursor()

        # Create buttons for each day of the week
        days = ["Monday", "Tuesday", "Wednesday", "Thursday", "Friday"]
        for day in days:
            button = tk.Button(self.root, text=day, command=lambda day=day: self.show_day_courses(day))
            button.pack()

    # Create a function to retrieve the course information for a given day
```

```
    def get_day_courses(self, day):
        self.cursor.execute("SELECT * FROM schedule WHERE day=?", (day,))
        return self.cursor.fetchall()

    # Create a function to update the label when a button is clicked
    def show_day_courses(self, day):
        courses = self.get_day_courses(day)
        courses_string = "\n".join([f"{course[0]} ({course[2]} -
{course[3]})" for course in courses])
        self.courses_label.config(text=courses_string)


class RegisterForm(tk.Tk): #u do this at the beginning w tk.tk
    def __init__(self):  #the rest "self" r the attributes
        super().__init__()

        course = StringVar()
        # Set up the GUI
        self.title("Registration Form")
        self.geometry("500x500")


        # Create labels and entry fields for course, email, and password
        self.email_label = tk.Label(self, text="Email:")
        self.email_entry = tk.Entry(self)
        self.password_label = tk.Label(self, text="Password:")
        self.password_entry = tk.Entry(self, show="*")
        self.label_course = tk.Label(self, text= "Course to enroll")
        self.course_entry = tk.Entry(self, text="Choose course")
```

```
82:         list1 = ['Boxing','Muay Thai','Judo'];
83:         self.droplist=OptionMenu(self,course, *list1)
84:         self.droplist.config(width=15)
85:         course.set('Select your course')
86:         self.droplist.place(x=190,y=175)
87:
88:         # Create a register button
89:         self.register_button = tk.Button(self, text="Register",
command=self.register_user)
90:         self.login_label = tk.Label(self, text="Already have an account?")
91:         self.login_button = tk.Button(self, text="Login",
command=self.go_to_login)
92:
93:
94:         # Add the widgets to the GUI
95:         self.email_label.pack()
96:         self.email_entry.pack()
97:         self.password_label.pack()
98:         self.password_entry.pack()
99:         self.register_button.pack()
100:         self.login_label.pack()
101:         self.login_button.pack()
102:         self.label_course.pack()
103:         self.course_entry.pack()
104:
105:     def register_user(self):
106:         # Get the inputted email and password
107:         entered_email = self.email_entry.get()
108:         entered_password = self.password_entry.get()
109:         entered_course = self.course_entry.get()
```

```
110:
111:         def final_registeration(entered_email, entered_password):
112:             # Hash the entered password
113:             hashed_password =
hashlib.sha256(entered_password.encode()).hexdigest()
114:
115:             # Connect to the SQLite database
116:             connection = sqlite3.connect("program.db")
117:             cursor = connection.cursor()
118:
119:             # Insert the user information into the database
120:             cursor.execute("INSERT INTO USERS (id, email, password, course)
VALUES (?, ?, ?, ?)", (str(uuid.uuid4()), entered_email, hashed_password,
entered_course))
121:             connection.commit()
122:
123:             # Show a message to confirm the registration
124:             tk.messagebox.showinfo("Registration Successful", "You have
successfully registered.")
125:
126:             # Close the database connection
127:             connection.close()
128:
129:
130:         if len(entered_password)<=20:
131:             if re.match("^(?=.*[0-9])(?=.*[a-z])(?=.*[A-Z](?=.*[^a-bA-B0-
9]))",entered_password):
132:                 # messagebox.showwarning("User: "+ entered_email +"
successfully registered")
133:                 final_registeration(entered_email, entered_password)
134:             else:
```

```
135:                 messagebox.showwarning("User","Enter valid password")
136:                 return False
137:             #regular expressions
138:
139:
140:         else:
141:             messagebox.showwarning("Invalid","Length try to exceed")
142:             return False
143:
144:         if len(entered_email)>7:
145:             if re.match("^([a-zA-Z0-9_\-\.]+)@([a-zA-Z0-9_\-\.]+)\.([a-zA-
Z]{2,5})$",entered_email):
146:                 return True
147:             else:
148:                 messagebox.showwarning("Alert","Invalid E-mail enter by
user")
149:                 return False
150:         else:
151:             messagebox.showwarning("Alert","Email length is too small")
152:
153:
154:
155:     def go_to_login(self):
156:         self.destroy()
157:         login_page = LoginForm() #this has to be the name of the class you
want to be redirected to
158: # use this code above to redirect to another page, use the class to specify
what page
159:
160:
```

```
161: ########################################    LOGIN
################################################

162: class LoginForm(tk.Tk):

163:     def __init__(self):

164:         super().__init__()

165:

166:

167:         # Set up the GUI

168:         self.title("Login Form")

169:         self.geometry("500x500")

170:

171:         # Create labels and entry fields for username and password

172:         self.email_label = tk.Label(self, text="Email:")

173:         self.email_entry = tk.Entry(self)

174:         self.password_label = tk.Label(self, text="Password:")

175:         self.password_entry = tk.Entry(self, show="*")

176:

177:         # Create a login button

178:         self.login_button = tk.Button(self, text="Login",
command=self.check_credentials)

179:

180:         self.register_label = tk.Label(self, text="Dont have an account?")

181:         self.register_button = tk.Button(self, text="Register",
command=self.go_to_register)

182:

183:         # Add the widgets to the GUI

184:         self.email_label.pack()

185:         self.email_entry.pack()

186:         self.password_label.pack()

187:         self.password_entry.pack()
```

```
188:         self.login_button.pack()
189:         self.register_label.pack()
190:         self.register_button.pack()
191:
192:
193:
194:     def check_credentials(self):
195:         # Get the inputted username and password
196:         entered_email = self.email_entry.get()
197:         entered_password = self.password_entry.get()
198:
199:         # Hash the entered password
200:         hashed_password =
hashlib.sha256(entered_password.encode()).hexdigest()
201:
202:         # Connect to the SQLite database
203:         connection = sqlite3.connect("program.db")
204:         cursor = connection.cursor()
205:
206:         # Query the database for the entered username and hashed password
207:         cursor.execute("SELECT * FROM users WHERE email=? AND password=?",
(entered_email, hashed_password))
208:         result = cursor.fetchone()
209:
210:         # Check if the query returned a result
211:         if result:
212:             print("Login successful!")
213:             print(result)
214:
215:             def go_to_timetable(self):
```

```
216:                 self.destroy()
217:                 timetable_page = TimetableApp(root)
218:                 timetable_page.mainloop()
219:
220:             #check if user is admin or normal user
221:             if result[4] == 'User':
222:                #take them to timetable page
223:                go_to_timetable(self)
224:             else:
225:                #take them to admin page
226:                pass
227:
228:             # Grant access to the user
229:
230:         else:
231:             print("Invalid username or password.")
232:
233:         # Close the database connection
234:         connection.close()
235:
236:     def go_to_register(self):
237:         self.destroy()
238:         register_page = RegisterForm()
239:
240:
241:
242:
243:
244: if __name__ == "__main__":
```

```
245:     form = LoginForm()
246:     form.mainloop()
247:
248: ########################################   ADMIN PAGE
##################################################
249:
250: class MainWindow(tk.Tk):
251:   def __init__(self):
252:     super().__init__()
253:     self.entered_email = tk.StringVar()
254:     self.entered_password = tk.StringVar()
255:     self._switch_to_login_page()
256:
257:   def _switch_to_login_page(self):
258:     for widget in self.winfo_children():
259:       widget.destroy()
260:
261:   def _login(self):
262:     if self.entered_email.get() == "0ADMIN@ADMIN.COM" and
self.entered_password.get() == "0Admin":
263:       self._switch_to_admin_page()
264:     else:
265:       self._switch_to_non_admin_page()
266:
267:   def _switch_to_admin_page(self):
268:     # create and display the admin page
269:     for widget in self.winfo_children():
270:       widget.destroy()
271:     tk.Label(self, text="Admin Page").pack()
272:
```

```
273:   def _switch_to_non_admin_page(self):
274:     # create and display the non-admin page
275:     for widget in self.winfo_children():
276:       widget.destroy()
277:     tk.Label(self, text="Non-Admin Page").pack()
278:
279: ########################MODIFICATION OF COURSE
################################################################
280: def create_table():
281:    cursor.execute('''CREATE TABLE IF NOT EXISTS schedule (name TEXT PRIMARY
KEY, day TEXT, start_time TIME, end_time TIME)''')
282:    conn.commit()
283:
284: def add_course(course_name, day, start_time, end_time):
285:     cursor.execute("INSERT INTO schedule VALUES (?,?,?,?)",(course_name,
day, start_time, end_time))
286:     conn.commit()
287:
288: # Create the main Tkinter window
289: root = tk.Tk()
290: root.title("Admin Control Panel")
291: create_table()
292:
293: # Create a frame for the course name, day, start time, and end time inputs
294: input_frame = tk.Frame(root)
295: input_frame.pack()
296: # Create labels and entry widgets for the course name, day, start time, and
end time
297: course_name_label = tk.Label(input_frame, text="Course Name:")
298: course_name_label.grid(row=0, column=0)
```

```
299: course_name_entry = tk.Entry(input_frame)
300: course_name_entry.grid(row=0, column=1)
301:
302: day_label = tk.Label(input_frame, text="Day:")
303: day_label.grid(row=1, column=0)
304: day_entry = tk.Entry(input_frame)
305: day_entry.grid(row=1, column=1)
306:
307: start_time_label = tk.Label(input_frame, text="Start Time:")
308: start_time_label.grid(row=2, column=0)
309: start_time_entry = tk.Entry(input_frame)
310: start_time_entry.grid(row=2, column=1)
311:
312: end_time_label = tk.Label(input_frame, text="End Time:")
313: end_time_label.grid(row=3, column=0)
314: end_time_entry = tk.Entry(input_frame)
315: end_time_entry.grid(row=3, column=1)
316:
317:  # Create a "Add Course" button
318: add_course_button = tk.Button(root, text="Add Course", command=lambda:
add_course(course_name_entry.get(), day_entry.get(), start_time_entry.get(),
end_time_entry.get()))
319: add_course_button.pack()
320:
321: root.mainloop()
322:
323:
324: if __name__ == "__main__":
325:     app = MainWindow()
326:     app.mainloop()
```

```
327:
328:
329: ##########################################  TIMETABLE
##########################################
330: class MainApp:
331:     def __init__(self, root):
332:         self.root = root
333:         self.root.title("Main Page")
334:
335:         # Create a button to show the timetable page
336:         button = tk.Button(self.root, text="Timetable",
command=self.show_timetable_page)
337:         button.pack()
338:
339:     def show_timetable_page(self):
340:         # Clear the contents of the root window
341:         for widget in self.root.winfo_children():
342:             widget.destroy()
343:
344:         # Create the timetable page
345:         timetable_app = TimetableApp(self.root)
346:
347: class TimetableApp():
348:
349:     def __init__(self, root):
350:         self.root = root
351:         self.root.title("Course Timetable")
352:         self.courses_label = tk.Label(self.root, text="", font=("Arial",
12))
353:         self.courses_label.pack()
```

```
354:
355:         # Connect to the database (or create it if it doesn't exist)
356:         self.conn = sqlite3.connect('program.db')
357:
358:         # Create a cursor object
359:         self.cursor = self.conn.cursor()
360:
361:         # Create buttons for each day of the week
362:         days = ["Monday", "Tuesday", "Wednesday", "Thursday", "Friday"]
363:         for day in days:
364:             button = tk.Button(self.root, text=day, command=lambda day=day:
self.show_day_courses(day))
365:             button.pack()
366:
367:     # Create a function to retrieve the course information for a given day
368:     def get_day_courses(self, day):
369:         self.cursor.execute("SELECT * FROM schedule WHERE day=?", (day,))
370:         return self.cursor.fetchall()
371:
372:     # Create a function to update the label when a button is clicked
373:     def show_day_courses(self, day):
374:         courses = self.get_day_courses(day)
375:         courses_string = "\n".join([f"{course[0]} ({course[2]} -
{course[3]})" for course in courses])
376:         self.courses_label.config(text=courses_string)
377:
378: root = tk.Tk()
379: app = MainApp(root)
380: root.mainloop()
381:
```

```
382: ######################################## RESERVATIONS
############################################################

383:

384: class ReservationSystem:

385:     def __init__(self, master):

386:         self.master = master

387:         master.title("Reservation System")

388:

389:         # create database connection

390:         self.conn = sqlite3.connect('program.db')

391:         self.c = self.conn.cursor()

392:

393:         # create table to hold reservation data

394:         self.c.execute('''CREATE TABLE IF NOT EXISTS reservations

395:                           (course text, name text, email text, phone text,
date text)''')

396:

397:         # create course selection menu

398:         self.course_label = tk.Label(master, text="Select a course:")

399:         self.course_label.grid(row=0, column=0)

400:         self.course_var = tk.StringVar(master)

401:         self.course_var.set("Judo")  # default value

402:         self.course_menu = tk.OptionMenu(master, self.course_var, "Judo",
"Muay Thai", "Boxing")

403:         self.course_menu.grid(row=0, column=1)

404:

405:         # create input fields

406:         self.name_label = tk.Label(master, text="Name:")

407:         self.name_label.grid(row=1, column=0)

408:         self.name_entry = tk.Entry(master)
```

```
409:         self.name_entry.grid(row=1, column=1)
410:
411:         self.email_label = tk.Label(master, text="Email:")
412:         self.email_label.grid(row=2, column=0)
413:         self.email_entry = tk.Entry(master)
414:         self.email_entry.grid(row=2, column=1)
415:
416:         self.phone_label = tk.Label(master, text="Phone:")
417:         self.phone_label.grid(row=3, column=0)
418:         self.phone_entry = tk.Entry(master)
419:         self.phone_entry.grid(row=3, column=1)
420:
421:         self.date_label = tk.Label(master, text="Date:")
422:         self.date_label.grid(row=4, column=0)
423:         self.date_entry = tk.Entry(master)
424:         self.date_entry.grid(row=4, column=1)
425:
426:         # create submit button
427:         self.submit_button = tk.Button(master, text="Submit",
command=self.submit)
428:         self.submit_button.grid(row=5, column=1)
429:
430:         # create a queue for each course
431:         self.judo_queue = []
432:         self.muay_thai_queue = []
433:         self.boxing_queue = []
434:
435:     def submit(self):
436:         course = self.course_var.get()
```

```
437:         name = self.name_entry.get()
438:         email = self.email_entry.get()
439:         phone = self.phone_entry.get()
440:         date = self.date_entry.get()
441:
442:         # check if input fields are not empty
443:         if name == '' or email == '' or phone == '' or date == '':
444:             return
445:
446:         # insert reservation data into database
447:         self.c.execute("INSERT INTO reservations VALUES (?, ?, ?, ?, ?)",
(course, name, email, phone, date))
448:         self.conn.commit()
449:
450:         # add reservation to the appropriate queue
451:         if course == 'Judo':
452:             self.judo_queue.append((name, email, phone, date))
453:         elif course == 'Muay Thai':
454:             self.muay_thai_queue.append((name, email, phone, date))
455:         elif course == 'Boxing':
456:             self.boxing_queue.append((name, email, phone, date))
457:
458:         # clear input fields
459:         self.name_entry.delete(0, tk.END)
460:         self.email_entry.delete(0, tk.END)
461:         self.phone_entry.delete(0, tk.END)
462:         self.date_entry.delete(0, tk.END)
463:
464:         # show success message
```

```
465:         success_label = tk.Label(self.master, text="Reservation
successful!")
466:         success_label.grid(row=6, column=1)
467:     def enqueue(self, course, name, email, phone, date):
468:         # add reservation to the appropriate queue
469:         if course == 'Judo':
470:             self.judo_queue.append((name, email, phone, date))
471:         elif course == 'Muay Thai':
472:             self.muay_thai_queue.append((name, email, phone, date))
473:         elif course == 'Boxing':
474:             self.boxing_queue.append((name, email, phone, date))
475:
476:     def dequeue(self, course):
477:         # get the next reservation from the appropriate queue
478:         if course == 'Judo':
479:             if len(self.judo_queue) > 0:
480:                 return self.judo_queue.pop(0)
481:             else:
482:                 return None
483:         elif course == 'Muay Thai':
484:             if len(self.muay_thai_queue) > 0:
485:                 return self.muay_thai_queue.pop(0)
486:             else:
487:                 return None
488:         elif course == 'Boxing':
489:             if len(self.boxing_queue) > 0:
490:                 return self.boxing_queue.pop(0)
491:             else:
492:                 return None
```

```
493:
494: root = tk.Tk()
495: reservation = ReservationSystem(root)
496: root.mainloop()
497:
498:
499: ############################################## ENROLLING
##############################################
500: from tkinter import ttk
501: from datetime import datetime
502:
503: # Connect to the SQLite database
504: conn = sqlite3.connect("program.db")
505:
506: # Create the transactions table if it doesn't exist
507: c = conn.cursor()
508: c.execute("""CREATE TABLE IF NOT EXISTS transactions (
509:             id INTEGER PRIMARY KEY,
510:             date TEXT,
511:             item TEXT,
512:             price REAL
513:             )""")
514: conn.commit()
515:
516: # Create the users table if it doesn't exist
517: c.execute("""CREATE TABLE IF NOT EXISTS usr (
518:             id INTEGER PRIMARY KEY,
519:             name TEXT,
520:             course TEXT,
```

```
521:             date_enrolled TEXT
522:             )""")
523: conn.commit()
524:
525: # Define global variables
526: courses = {
527:     "Judo": 99.99,
528:     "Muay Thai": 79.99,
529:     "Boxing": 69.99
530: }
531:
532: # Define the main window
533: root = tk.Tk()
534: root.title("Enrollment System")
535:
536: # Define functions for handling enrollments
537: def enroll_user():
538:     # Get the user's name and selected course from the GUI
539:     name = name_var.get()
540:     course = course_var.get()
541:
542:     # Get the price of the selected course
543:     price = courses[course]
544:
545:     # Add a transaction to the database
546:     date = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
547:     c.execute("INSERT INTO transactions (date, item, price) VALUES (?, ?,
?)", (date, course, price))
548:     conn.commit()
```

```
549:
550:     # Add the user to the database
551:     date_enrolled = datetime.now().strftime("%Y-%m-%d")
552:     c.execute("INSERT INTO usr (name, course, date_enrolled) VALUES (?, ?,
?)", (name, course, date_enrolled))
553:     conn.commit()
554:
555:     # Generate a receipt
556:     receipt_text = f"Enrollment Receipt\n\nName: {name}\nCourse:
{course}\nPrice: ${price:.2f}\nDate: {date}\n\nThank you for enrolling!"
557:     receipt_label.config(text=receipt_text)
558:
559: # Define functions for handling billing
560: def calculate_total():
561:     # Get the selected course from the GUI
562:     course = course_var.get()
563:
564:     # Get the price of the selected course
565:     price = courses[course]
566:
567:     # Calculate the total price based on the quantity
568:     quantity = int(quantity_var.get())
569:     total = price * quantity
570:
571:     # Update the total label on the GUI
572:     total_label.config(text=f"Total: ${total:.2f}")
573:
574: # Create labels and entry widgets for the GUI
575: name_label = tk.Label(root, text="Name:")
576: name_var = tk.StringVar()
```

```
577: name_entry = tk.Entry(root, textvariable=name_var)
578:
579: course_label = tk.Label(root, text="Course:")
580: course_var = tk.StringVar()
581: course_dropdown = ttk.Combobox(root, textvariable=course_var,
values=list(courses.keys()), state="readonly", postcommand=calculate_total)
582:
583: quantity_label = tk.Label(root, text="Quantity:")
584: quantity_var = tk.StringVar(value="1")
585: quantity_entry = tk.Entry(root, textvariable=quantity_var, validate="key",
validatecommand=(root.register(lambda s: s.isdigit() or s == ""), "%S"), width=3)
586: quantity_entry.bind("<KeyRelease>", lambda e: calculate_total())
587:
588: total_label = tk.Label(root, text="Total: $0.00")
589:
590: # Create a button for the GUI to enroll the user
591: enroll_button = tk.Button(root, text="Enroll", command=enroll_user)
592:
593: # Create a label for the receipt
594: receipt_label = tk.Label(root, text="")
595:
596: # Pack the widgets into the GUI
597: name_label.grid(row=0, column=0, padx=5, pady=5, sticky=tk.E)
598: name_entry.grid(row=0, column=1, padx=5, pady=5, sticky=tk.W)
599:
600: course_label.grid(row=1, column=0, padx=5, pady=5, sticky=tk.E)
601: course_dropdown.grid(row=1, column=1, padx=5, pady=5, sticky=tk.W)
602:
603: quantity_label.grid(row=2, column=0, padx=5, pady=5, sticky=tk.E)
604: quantity_entry.grid(row=2, column=1, padx=5, pady=5, sticky=tk.W)
```

```
605:
606: total_label.grid(row=2, column=2, padx=5, pady=5, sticky=tk.E)
607:
608: enroll_button.grid(row=3, column=1, padx=5, pady=5)
609:
610: receipt_label.grid(row=4, column=0, columnspan=3, padx=5, pady=5)
611:
612: root.mainloop()
613:
614:
615: ####################################card
transaction#########################################################
616:
617:
618: class CardTransactionSystem:
619:     def __init__(self, master):
620:         self.master = master
621:         master.title("Card Transaction System")
622:
623:         # create database connection
624:         self.conn = sqlite3.connect('program.db')
625:         self.c = self.conn.cursor()
626:
627:         # create table to hold transaction data
628:         self.c.execute('''CREATE TABLE IF NOT EXISTS transac
629:                           (card_number text, amount real, date text)''')
630:
631:         # create input fields
632:         self.card_number_label = tk.Label(master, text="Card Number:")
```

```
633:         self.card_number_label.grid(row=0, column=0)
634:         self.card_number_entry = tk.Entry(master)
635:         self.card_number_entry.grid(row=0, column=1)
636:
637:         self.amount_label = tk.Label(master, text="Amount:")
638:         self.amount_label.grid(row=1, column=0)
639:         self.amount_entry = tk.Entry(master)
640:         self.amount_entry.grid(row=1, column=1)
641:
642:         self.date_label = tk.Label(master, text="Date:")
643:         self.date_label.grid(row=2, column=0)
644:         self.date_entry = tk.Entry(master)
645:         self.date_entry.grid(row=2, column=1)
646:
647:         # create submit button
648:         self.submit_button = tk.Button(master, text="Submit",
command=self.submit)
649:         self.submit_button.grid(row=3, column=1)
650:
651:     def submit(self):
652:         card_number = self.card_number_entry.get()
653:         amount = float(self.amount_entry.get())
654:         date = self.date_entry.get()
655:
656:         # insert transaction data into database
657:         self.c.execute("INSERT INTO transac VALUES (?, ?, ?)", (card_number,
amount, date))
658:         self.conn.commit()
659:
660:         # clear input fields
```

```
661:         self.card_number_entry.delete(0, tk.END)
662:         self.amount_entry.delete(0, tk.END)
663:         self.date_entry.delete(0, tk.END)
664:
665:     def __del__(self):
666:         # close database connection
667:         self.conn.close()
668:
669: root = tk.Tk()
670: card_transaction_system = CardTransactionSystem(root)
671: root.mainloop()
```